APLICACIONES WEB 2

Kinesio.Saludd

Sistema web para consultorio de kinesiología

Sebastian De Diego

Comisión MB

IES Córdoba

2026

1. Descripción del proyecto

Kinesio.Saludd es un proyecto web pensado para un consultorio de kinesiología y fisioterapia ubicado en Río Primero, Córdoba. El sitio reúne la información principal del consultorio, los servicios disponibles y las profesionales que trabajan en el lugar.

Además de la parte informativa, se agregó una función para solicitar turnos desde la misma página. De esta forma el sitio no queda solamente como una presentación del consultorio, sino que empieza a resolver una necesidad concreta de atención y organización.

2. Necesidad detectada

El consultorio necesitaba contar con un espacio web donde los pacientes pudieran encontrar la información principal de una manera sencilla y ordenada.

También se planteó la necesidad de mejorar la forma de solicitar turnos. Para eso se incorporó un formulario que permite registrar los datos del paciente, la fecha, el horario y la profesional elegida.

3. Objetivo general

Desarrollar un sitio web para Kinesio.Saludd que permita mostrar información del consultorio y sus servicios, facilitar el contacto con los pacientes y registrar solicitudes de turnos.

4. Objetivos específicos

- Mostrar la información principal del consultorio.
- Presentar los servicios de kinesiología y fisioterapia.
- Mostrar información de las profesionales.
- Facilitar el acceso a los datos de contacto.
- Permitir que un paciente solicite un turno.
- Guardar los datos de los turnos solicitados.
- Consultar los turnos registrados desde el servidor.
- Mantener una página adaptable a distintos tamaños de pantalla.

5. Alcance del sistema

En su estado actual el sistema permite navegar por las páginas principales del consultorio, consultar información de los servicios y las profesionales, acceder a la sección de contacto y solicitar un turno.

Para cada turno se registran nombre, teléfono, fecha, hora y profesional. Los datos se guardan utilizando MockAPI y pueden consultarse desde una ruta del servidor.

6. Usuarios

El usuario principal del sitio es el paciente. Puede recorrer la página, consultar la información disponible y solicitar un turno.

Por el momento no se desarrolló un panel interno para las profesionales. La parte implementada está enfocada en el uso del paciente desde la página web.

7. Funcionalidades principales

Información del consultorio

La página muestra información general de Kinesio.Saludd, su ubicación y una descripción del espacio.

Servicios

Se presenta una sección con los tratamientos y servicios disponibles, entre ellos kinesiología y fisioterapia, rehabilitación traumatológica, kinesioterapia respiratoria, masoterapia, osteopatía y punción seca.

Profesionales

En la página de inicio se muestran las profesionales que forman parte del consultorio junto con una breve descripción.

Contacto y turnos

La sección de contacto incluye información del consultorio, un acceso directo a WhatsApp y un formulario para solicitar turnos. Antes de enviar el formulario se controla que los campos obligatorios estén completos.

8. Tecnologías utilizadas

- HTML para la estructura de las páginas.
- CSS para el diseño y la adaptación visual.
- JavaScript para el funcionamiento del formulario de turnos.
- Node.js para ejecutar el servidor.
- Express para servir la página y crear las rutas.
- MockAPI para almacenar los turnos.
- Git y GitHub para guardar y organizar el proyecto.
- pnpm para instalar y manejar las dependencias.

9. Estructura del proyecto

El proyecto se encuentra dentro de la carpeta TP2. La carpeta web contiene la parte visible del sitio y el archivo index.mjs contiene el servidor realizado con Express.

```
TP2/
├── index.mjs
├── package.json
├── pnpm-lock.yaml
├── README.md
└── web/
    ├── index.html
```

```
├── servicios.html
├── contacto.html
└── recursos/
    ├── css/
    ├── imagenes/
    └── js/
```

10. Servidor web

Se incorporó un servidor utilizando Node.js y Express. Express permite servir los archivos que se encuentran dentro de la carpeta web, por lo que las páginas HTML, los estilos, las imágenes y los archivos JavaScript se cargan desde el mismo servidor.

El proyecto se ejecuta de manera local en http://localhost:3000.

11. Gestión de turnos

Para guardar los turnos se creó un recurso llamado turnos en MockAPI. Cada registro almacena el nombre del paciente, teléfono, fecha, hora y profesional.

El formulario de la página de contacto toma los datos ingresados, los envía al servidor y desde allí se realiza la conexión con MockAPI.

12. Rutas utilizadas

GET /api/turnos

Esta ruta se utiliza para consultar los turnos guardados. El servidor obtiene la información desde MockAPI y la devuelve en formato JSON.

POST /api/turnos

Esta ruta se utiliza para registrar un turno nuevo. Recibe los datos cargados desde el formulario y los envía a MockAPI para que queden almacenados.

13. Funcionamiento del formulario de turnos

Cuando el paciente completa el formulario y presiona el botón para solicitar un turno, JavaScript toma los datos ingresados.

Antes de enviar la información se verifica que nombre, teléfono, fecha, hora y profesional estén completos. Si falta algún dato se muestra un mensaje. Si todos los campos están completos, se realiza una petición POST a /api/turnos.

Cuando el turno se guarda correctamente se muestra un mensaje de confirmación y el formulario vuelve a quedar vacío.

14. Diseño y adaptación

Se mantuvo la misma línea visual del sitio utilizando tonos verdes y fondos claros. También se reorganizó la página de contacto para evitar tener dos formularios con funciones similares.

Se dejó una sección informativa y un único formulario para solicitar turnos. Además, el diseño incluye reglas responsive para que la página se adapte a pantallas más pequeñas.

15. Cronograma de desarrollo

El trabajo se organizó en pasos más específicos para poder seguir mejor el avance del proyecto.

Etapa	Tareas realizadas
1. Análisis inicial	Definición de la idea, necesidad del consultorio, usuarios y objetivos.
2. Organización del contenido	Definición de páginas, servicios, imágenes e información de las profesionales.
3. Interfaz inicial	Creación de inicio, servicios, contacto, navegación y adaptación responsive.
4. Preparación del backend	Instalación de Node.js, pnpm y Express. Creación y prueba del servidor local.
5. Archivos estáticos	Configuración de Express para servir la carpeta web y comprobación de las páginas.
6. MockAPI	Creación del recurso turnos y definición de los campos necesarios.
7. Consulta de turnos	Creación y prueba de GET /api/turnos.
8. Registro de turnos	Creación y prueba de POST /api/turnos.
9. Formulario	Creación del formulario de turno y conexión con el backend.
10. Validaciones	Control de campos obligatorios y corrección de datos incompletos.
11. Ajustes visuales	Reorganización de contacto y mejora de estilos del turnero.
12. Repositorio	Configuración de .gitignore, commits, README y subida de cambios a GitHub.

16. Estado actual

Actualmente el sitio permite navegar por las páginas principales y registrar solicitudes de turnos. El frontend y el backend ya se encuentran conectados y los turnos quedan almacenados en MockAPI.

17. Próximos pasos

- Mostrar los turnos en una pantalla interna.
- Permitir modificar o cancelar turnos.
- Evitar reservas duplicadas para el mismo horario.
- Agregar acceso para las profesionales.
- Administrar horarios disponibles.

El proyecto se encuentra en el repositorio de trabajos prácticos de Aplicaciones Web 2, dentro de la carpeta TP2.

18. TP3 - Entrega parcial (punto 3.3)

Fecha: 16 de octubre de 2026

18.1. Objetivo

En esta etapa se incorporo una interfaz de administracion para gestionar los servicios del consultorio Kinesio.Saludd. Se continuo el mismo proyecto del TP2, conservando la pagina publica, el servidor Express y las rutas de turnos.

18.2. Funcionalidades desarrolladas

- Listado de servicios y busqueda por nombre.
- Formulario para agregar nuevos servicios.
- Edicion de nombre, descripcion y estado de cada servicio.
- Eliminacion de servicios con confirmacion.
- Contadores de servicios totales y activos.

18.3. Archivos incorporados

- TP3/web/admin/index.html - estructura del panel.
- TP3/web/admin/estilos-admin.css - estilos de la interfaz.
- TP3/web/admin/admin.js - manejo de eventos y datos.

18.4. Estado de la entrega parcial

Para esta entrega se desarrollo y probo la interfaz de administracion. Los datos son de ejemplo y se almacenan temporalmente en localStorage del navegador. Las operaciones no estan conectadas aun a una API de servicios ni a una base de datos.

La integracion con endpoints CRUD, la organizacion modular MVC, la base de datos y la documentacion de API quedan pendientes para la entrega final del TP3.

18.5. Comprobaciones

Se comprobo en el navegador la carga del panel, el alta, la modificacion y la baja de servicios, junto con la actualizacion de los contadores. La interfaz utiliza HTML, CSS y JavaScript sin frameworks adicionales.

19. TP3 - Entrega final

Fecha: 23 de octubre de 2026. Se continuó el trabajo realizado en la entrega parcial del 16 de octubre. En esa instancia el panel guardaba datos de ejemplo en localStorage; en la entrega final se lo conectó a una API y a una base de datos PostgreSQL.

19.1. Objetivo

Completar la administración de servicios del consultorio Kinesio.Saludd. Se mantuvo el sitio público y el sistema de turnos del TP2, y se incorporó la persistencia de los servicios mediante una base de datos.

19.2. Organización del proyecto (MVC y Router)

Para organizar el código se creó el módulo **servicios**, con archivos separados. El modelo realiza las consultas SQL a PostgreSQL; el controlador recibe las peticiones, controla los datos y prepara las respuestas; y Router define las rutas de la API. El archivo index.mjs inicia Express y vincula las rutas del módulo.

Archivo	Función
servicios/modelo.servicios.mjs	Consultas SELECT, INSERT, UPDATE y DELETE.
servicios/controlador.servicios.mjs	Recibe las peticiones y responde en JSON.
servicios/rutas.servicios.mjs	Define los cinco endpoints mediante Router.
bd/conexion.mjs	Conexión a PostgreSQL con el paquete pg.
index.mjs	Configura y levanta el servidor Express.

19.3. Base de datos y Docker

Se utilizó **PostgreSQL** para guardar los servicios. Para ejecutarlo localmente se instaló Docker Desktop y se preparó un Dockerfile que toma la imagen de PostgreSQL e incorpora el archivo servicios.sql. La base se llama **kinesio_saludd** y contiene una tabla servicios con los campos id, nombre, descripcion y activo.

Los datos de conexión se guardan en un archivo .env local, que está excluido del repositorio mediante .gitignore. El script bd/servicios.sql permite crear la tabla y agregar los datos iniciales de demostración.

19.4. API de servicios

Método y ruta	Uso
GET /api/servicios	Obtener el listado completo de servicios.
GET /api/servicios/:id	Consultar un servicio por su identificador.

Método y ruta	Uso
POST /api/servicios	Agregar un servicio nuevo.
PUT /api/servicios/:id	Modificar nombre, descripción y estado.
DELETE /api/servicios/:id	Eliminar un servicio.

Las rutas reciben o devuelven datos JSON. La API utiliza consultas SQL parametrizadas y comprueba los campos obligatorios antes de guardar. Como el frontend y la API se ejecutan desde el mismo dominio y puerto local, no se necesitó habilitar CORS para estas pruebas.

19.5. Interfaz administrativa y sitio público

El panel de administración está disponible en **/admin/**. Mantiene el listado, la búsqueda, los contadores y los formularios de alta y modificación de la entrega parcial. La diferencia es que ahora utiliza fetch para comunicarse con la API, y los cambios quedan almacenados en PostgreSQL en lugar de localStorage.

La sección pública de servicios también consulta la API y muestra los servicios que tienen el estado activo. Se conserva la parte de turnos de TP2: sus endpoints siguen utilizando MockAPI.

19.6. Documentación con Scalar

Se incorporó documentación de la API mediante Scalar, disponible en **http://localhost:3000/api-docs/** al ejecutar el servidor local. La documentación describe las cinco rutas, los métodos utilizados y los datos de entrada y salida.

19.7. Pruebas realizadas

Durante la prueba local se ejecutó PostgreSQL desde Docker. Se consultó la tabla servicios con SELECT y se verificó que tenía seis registros iniciales. Luego se inició Express y se comprobó que GET /api/servicios devolvía el listado completo en JSON.

Desde el panel administrativo se agregó un servicio de prueba, se modificaron sus datos y su estado, y finalmente se eliminó. Los contadores reflejaron los cambios y el listado volvió a los seis servicios originales. También se comprobó que GET /api/servicios/1 devolvía un único registro y que la documentación de Scalar se abría correctamente.

19.8. Estructura y archivos para la entrega

Ubicación	Contenido
TP3/web	Sitio público y panel administrativo.
TP3/servicios	Modelo, controlador y rutas.
TP3/bd/servicios.sql	Script SQL para recrear la tabla y datos iniciales.
TP3/bd/Dockerfile	Configuración local de PostgreSQL con Docker.

Ubicación	Contenido
TP3/web/api-docs	Documentación OpenAPI y Scalar.
TP3/documentacion	Informe acumulativo TP1, TP2 y TP3.

19.9. Estado final y próximas mejoras

Se completaron y probaron las cinco operaciones del CRUD de servicios, la base de datos PostgreSQL, el panel administrativo y la documentación de la API. Como mejora posterior queda la autenticación de usuarios y la protección del panel, contenido previsto para el TP4. Por ese motivo el panel actual es una implementación de práctica para uso local.

Repositorio: https://github.com/sebadediego/aplicaciones-web-2-tps